

NPTEL Course on
"Computer System Optimization for ML"

Lecture #35

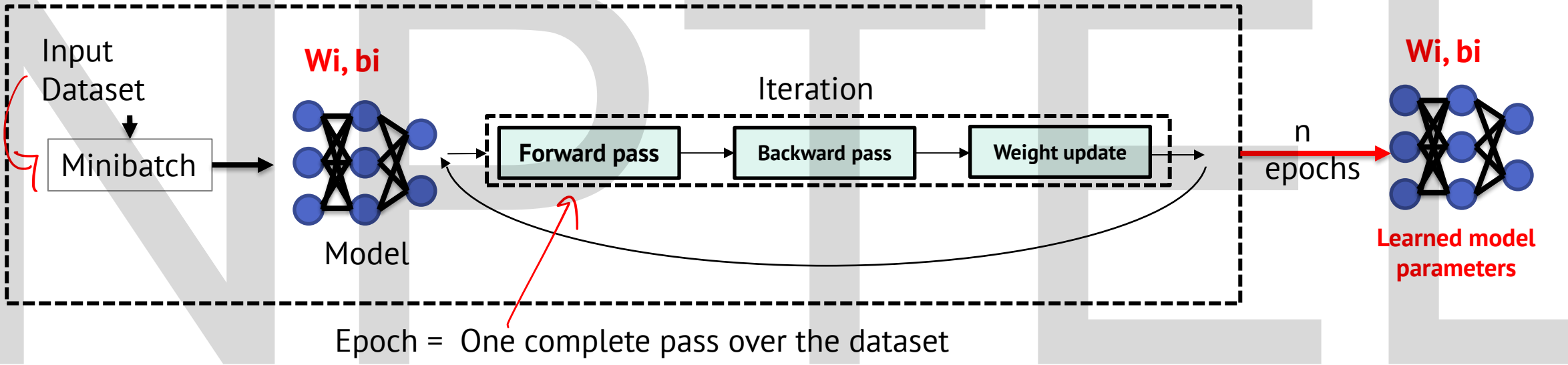
NPTEL

Agenda

- **Fault Tolerance in Machine Learning**

NPTEL

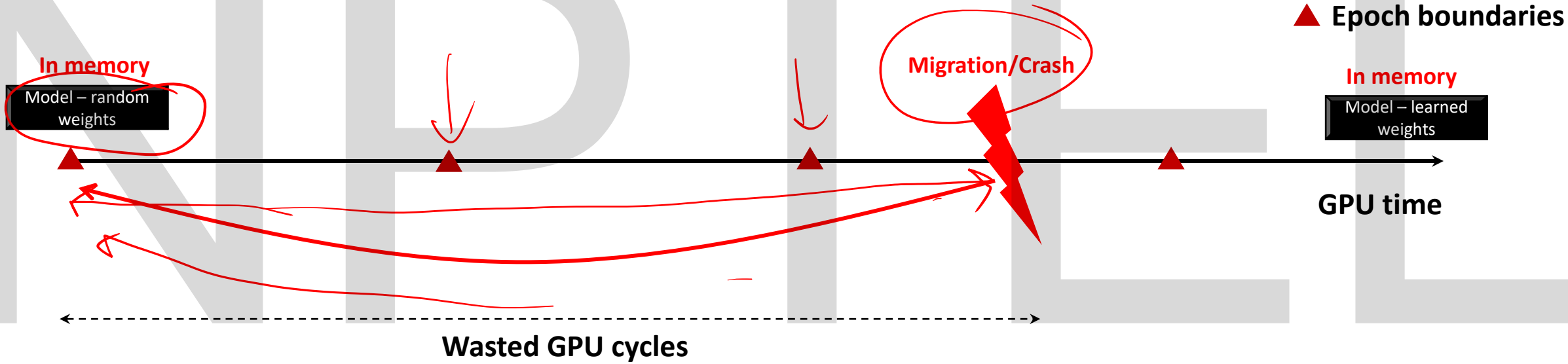
DNN Training Revisited



DNN training is compute-intensive and time-consuming!

DNN Checkpointing

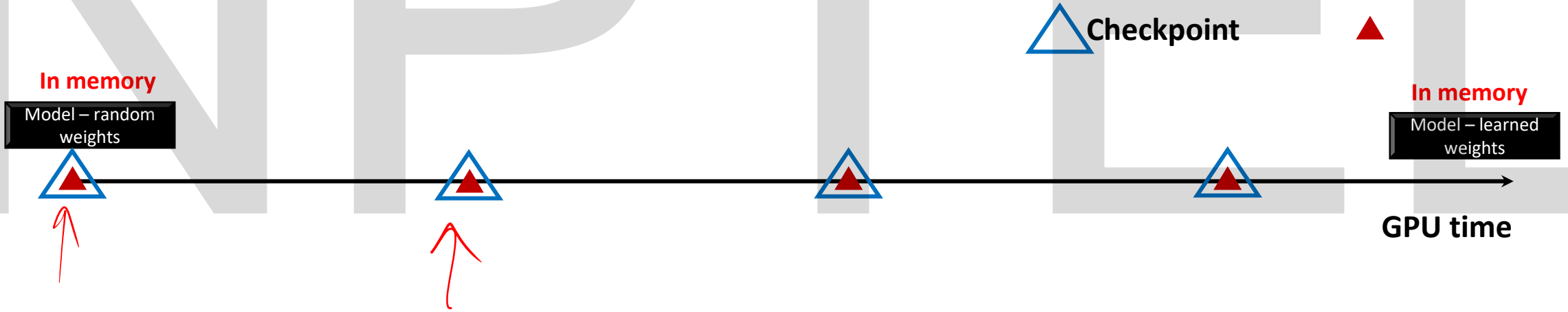
Any interruption can wipe out the model parameters learned so far in memory, restarting this expensive process!



NPTEL

DNN Checkpointing

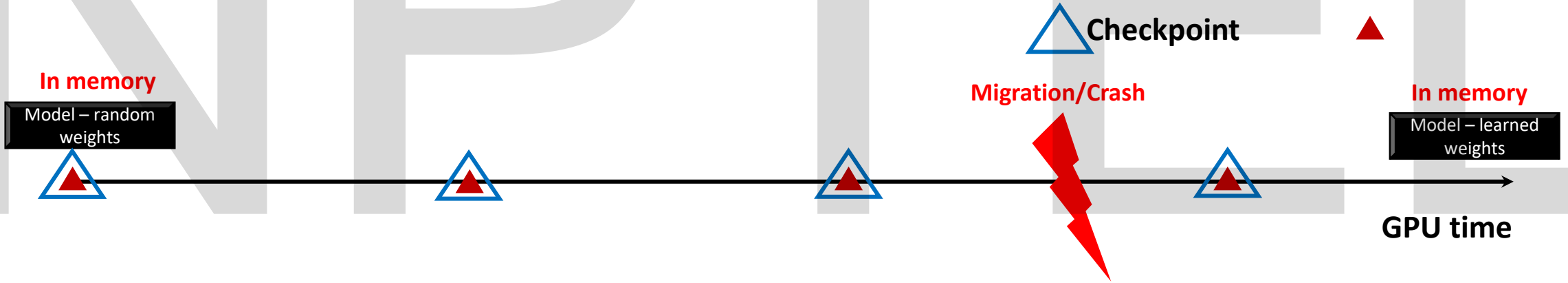
- Learned model parameters are written to persistent storage every so often during training for fault-tolerance
  - The VMs may migrate, expire or crash, jobs may migrate (e.g., shared GPU clusters)



NPTEL

DNN Checkpointing

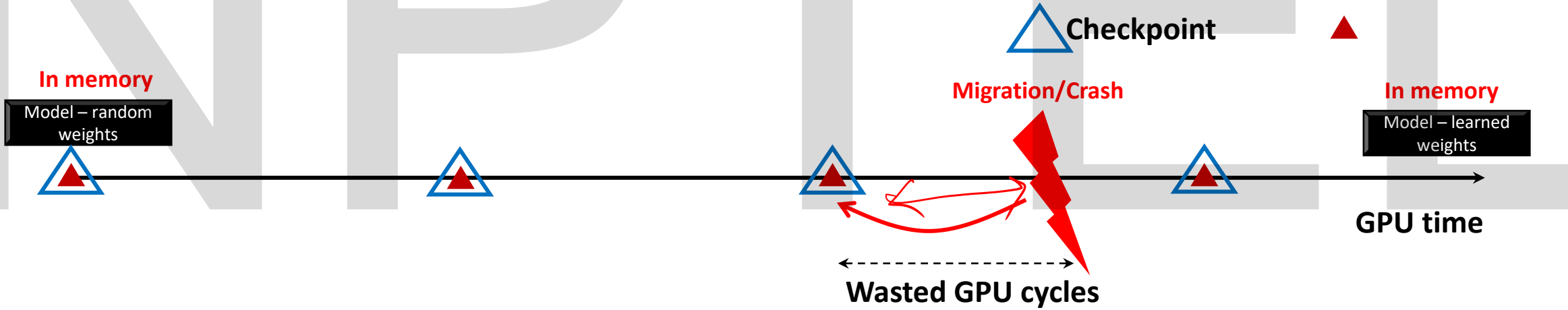
- Learned model parameters are written to persistent storage every so often during training for fault-tolerance
  - The VMs may migrate, expire or crash, jobs may migrate (e.g., shared GPU clusters)



NPTEL

DNN Checkpointing

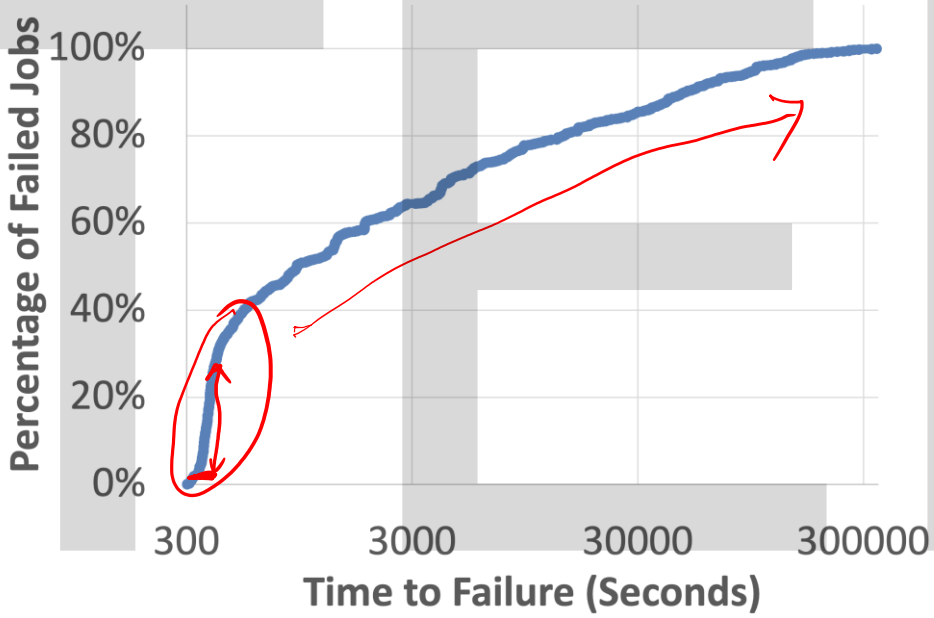
- Learned model parameters are written to persistent storage every so often during training for fault-tolerance
  - The VMs may migrate, expire or crash, jobs may migrate (e.g., shared GPU clusters)



NPTEL

The Criticality of Checkpointing

- Failure Recovery (ensure progress)
- Migrating training jobs
- Publishing snapshots
- Transfer learning



NPTEL

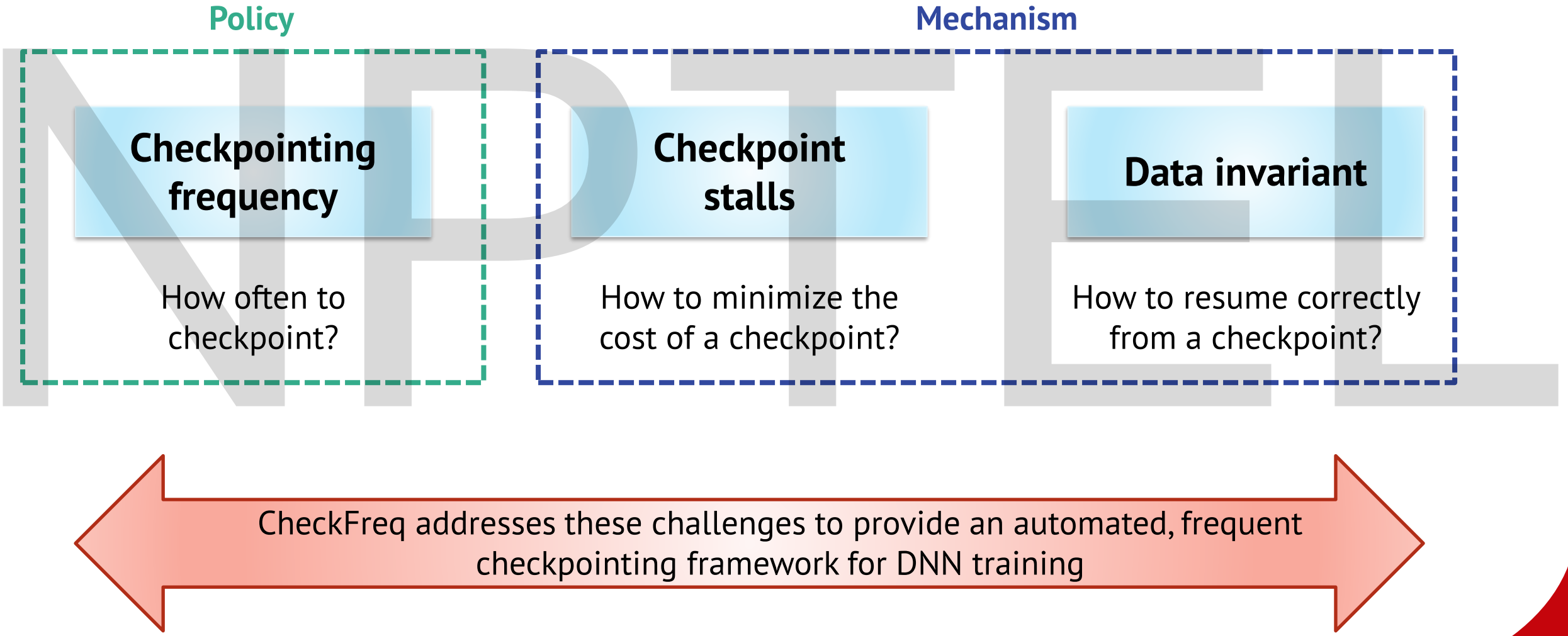
CheckFreq (FAST'21)

- Synchronous checkpoints => large checkpoint stalls
- Manual checkpointing frequency => typically performed at epoch boundaries
  - But model epoch times are increasing due to higher computational complexity and increasing dataset sizes
  - Frequent interruptions: for e.g., pre-emptions in low-cost spot VMs

Need fine-grained, iteration-level checkpointing

NPTEL

Challenges for Fine-Grained Checkpointing



CheckFreq Design

Mechanism

How to perform correct, low-cost checkpointing?

2-phase DNN-aware checkpointing

Low checkpoint stalls

Resumable data iterator

Maintain data invariant

Policy

When to checkpoint?

Systematic online profiling

Initial checkpointing frequency

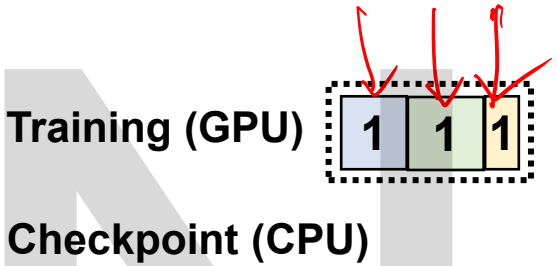
Adaptive rate tuning

Manages interference from other jobs

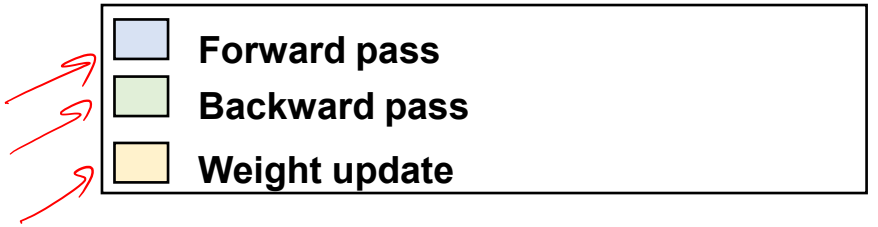
NPTEL

Example

- **Consider a policy that checkpoints every three iterations**

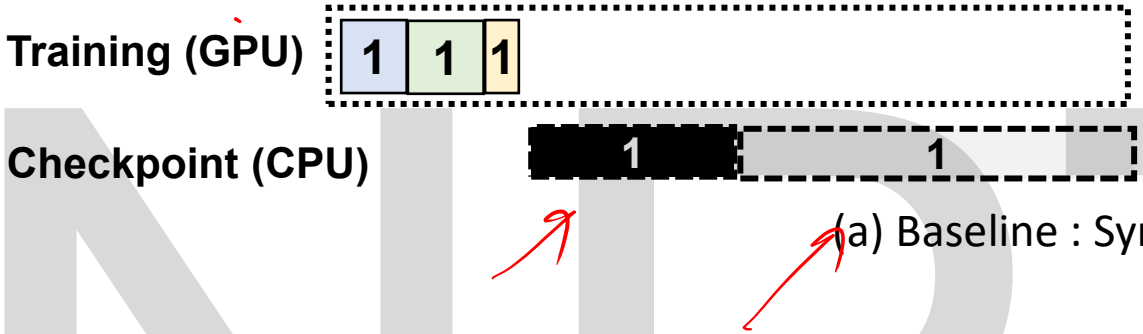


(a) Baseline : Synchronous checkpointing

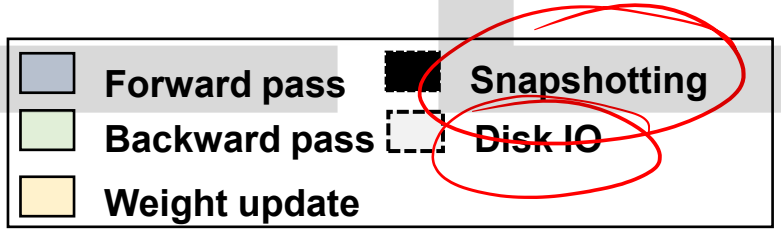


NPTEL

Example

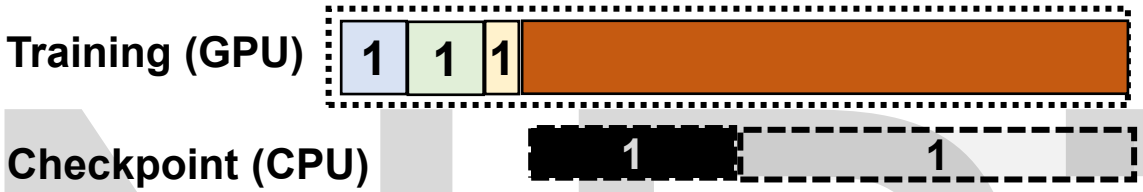


(a) Baseline : Synchronous checkpointing

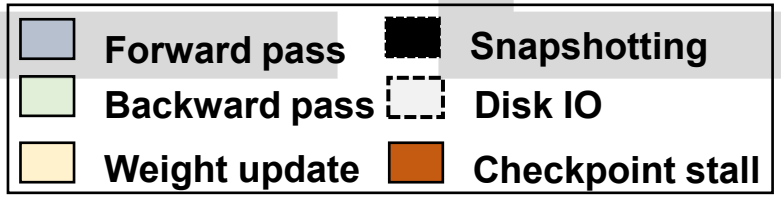


NPTEL

Example

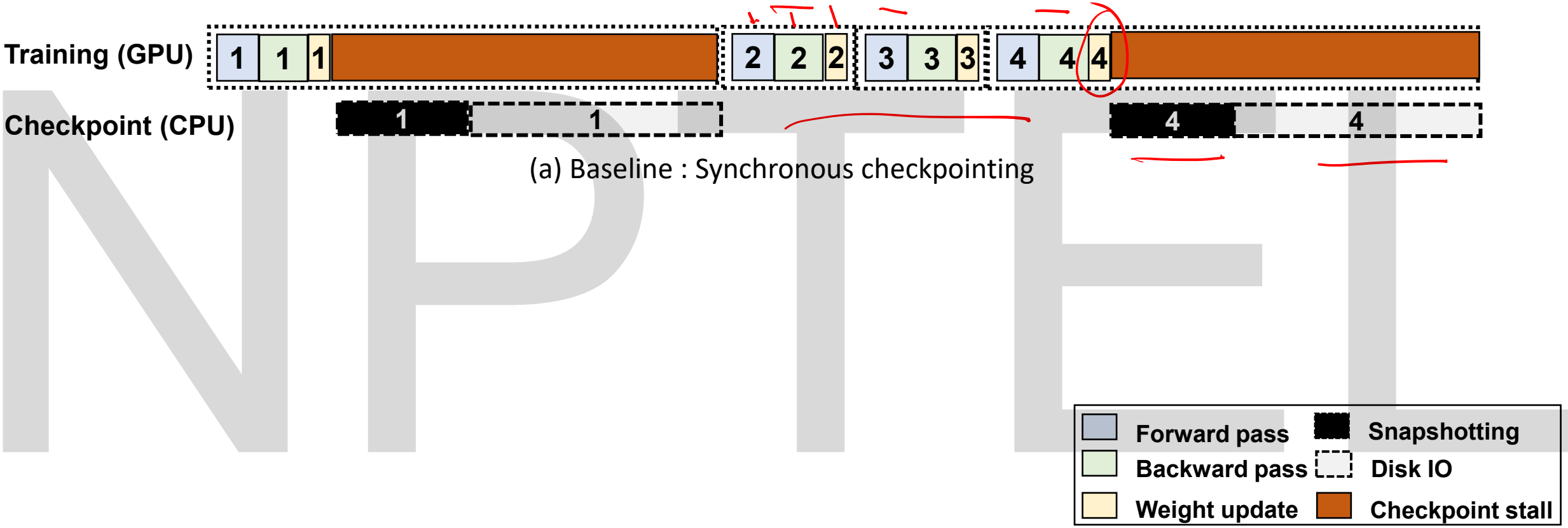


(a) Baseline : Synchronous checkpointing



NPTEL

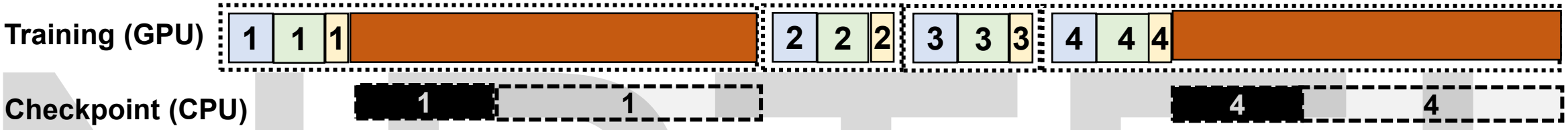
Example



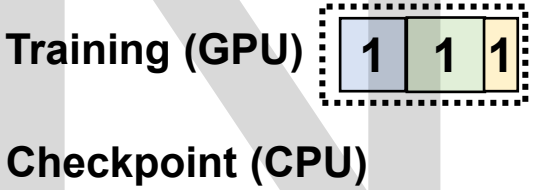
(a) Baseline : Synchronous checkpointing

NPTEL

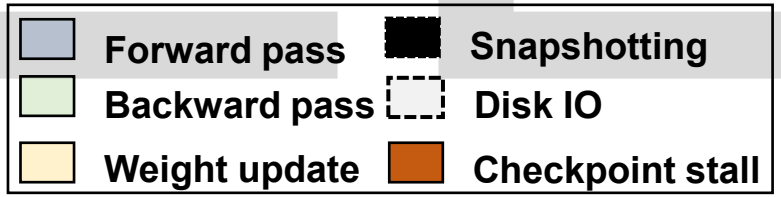
Example



(a) Baseline : Synchronous checkpointing

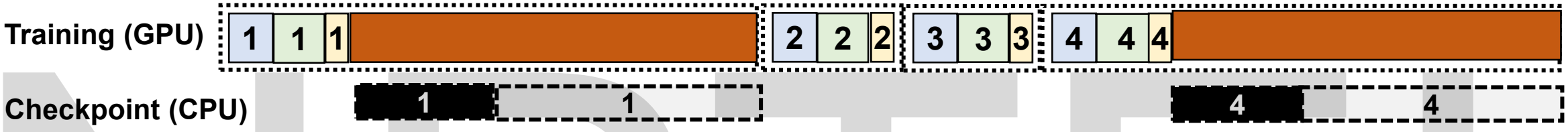


(b) Only persist() pipelining

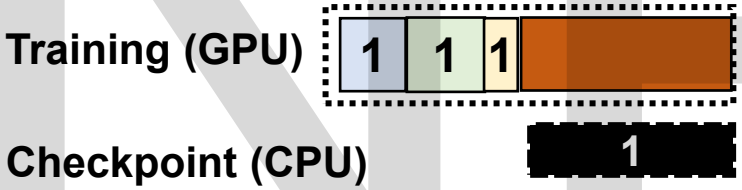


NPTEL

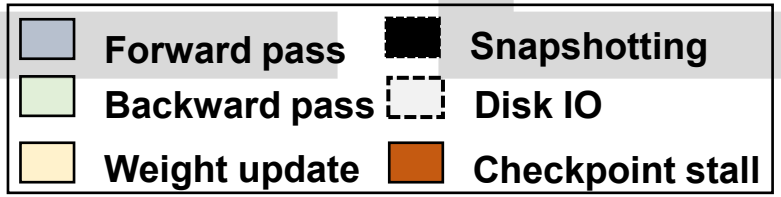
Example



(a) Baseline : Synchronous checkpointing

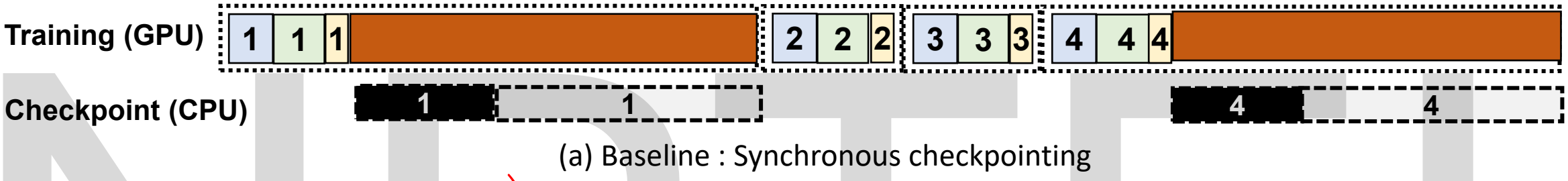


(b) Only persist() pipelining

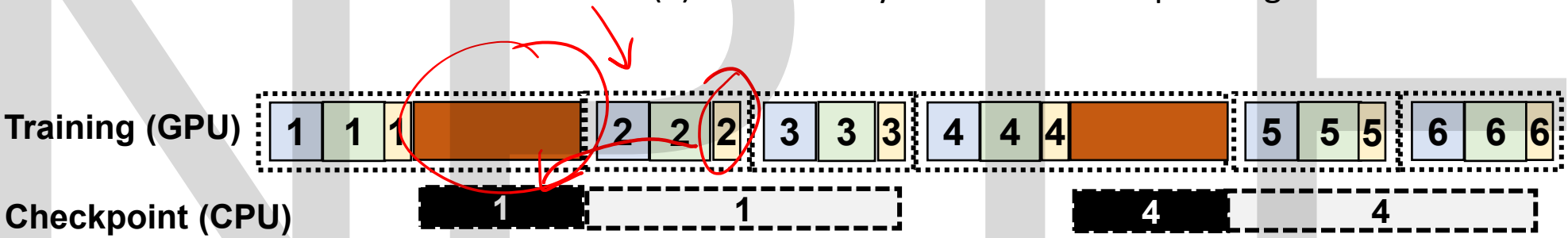


NPTEL

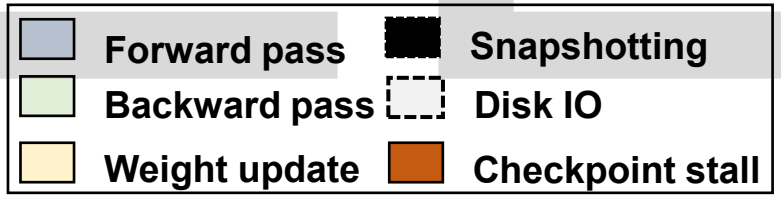
Example



(a) Baseline : Synchronous checkpointing

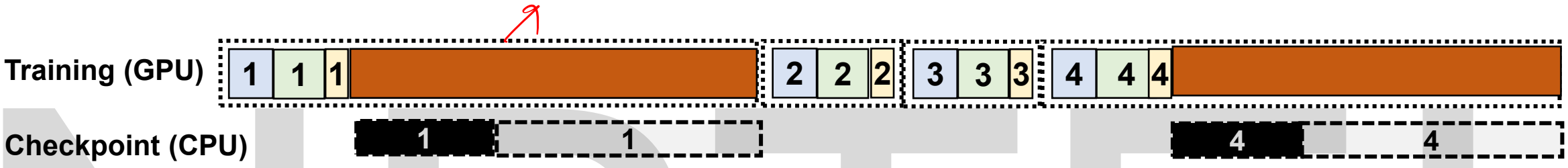


(b) Only persist() pipelining

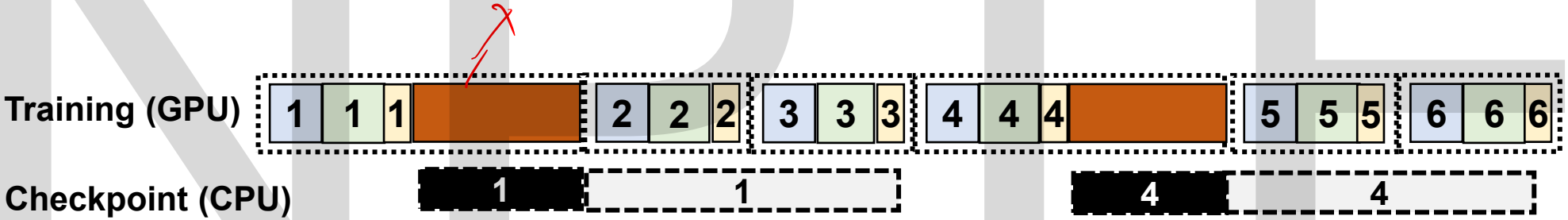


NRTL

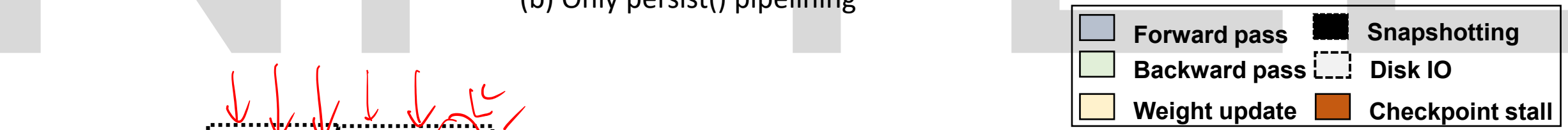
Example

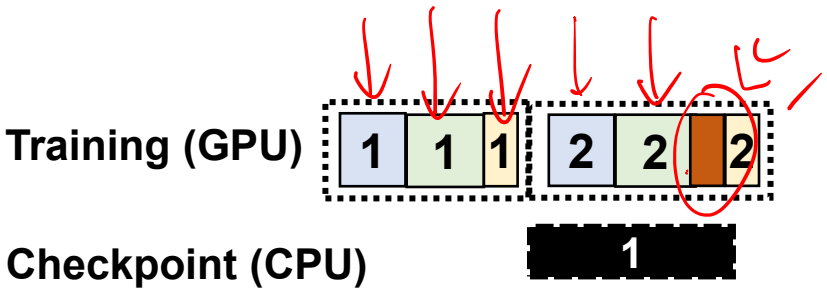


(a) Baseline : Synchronous checkpointing



(b) Only persist() pipelining

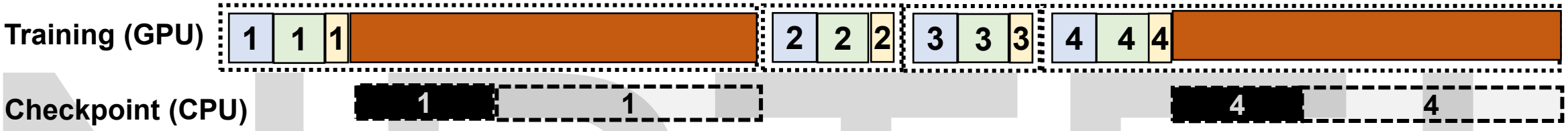




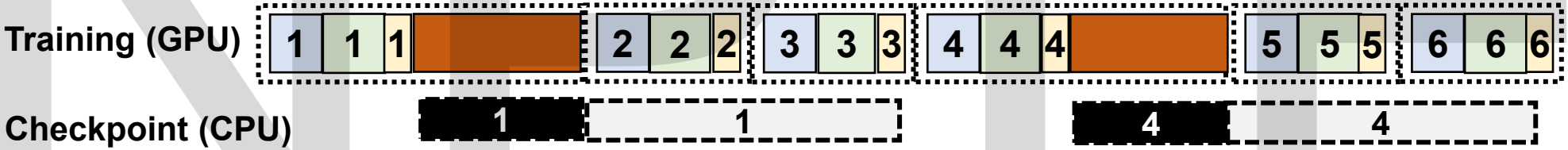
(c) Snapshot() and persist() pipelining

NPTEL

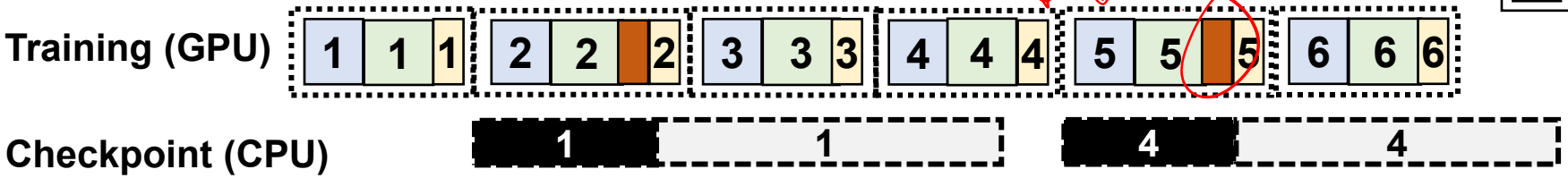
Example



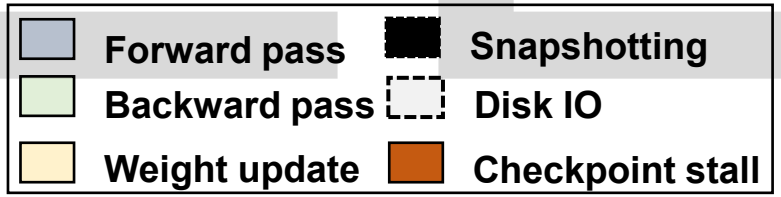
(a) Baseline : Synchronous checkpointing



(b) Only persist() pipelining



(c) Snapshot() and persist() pipelining



NDTF

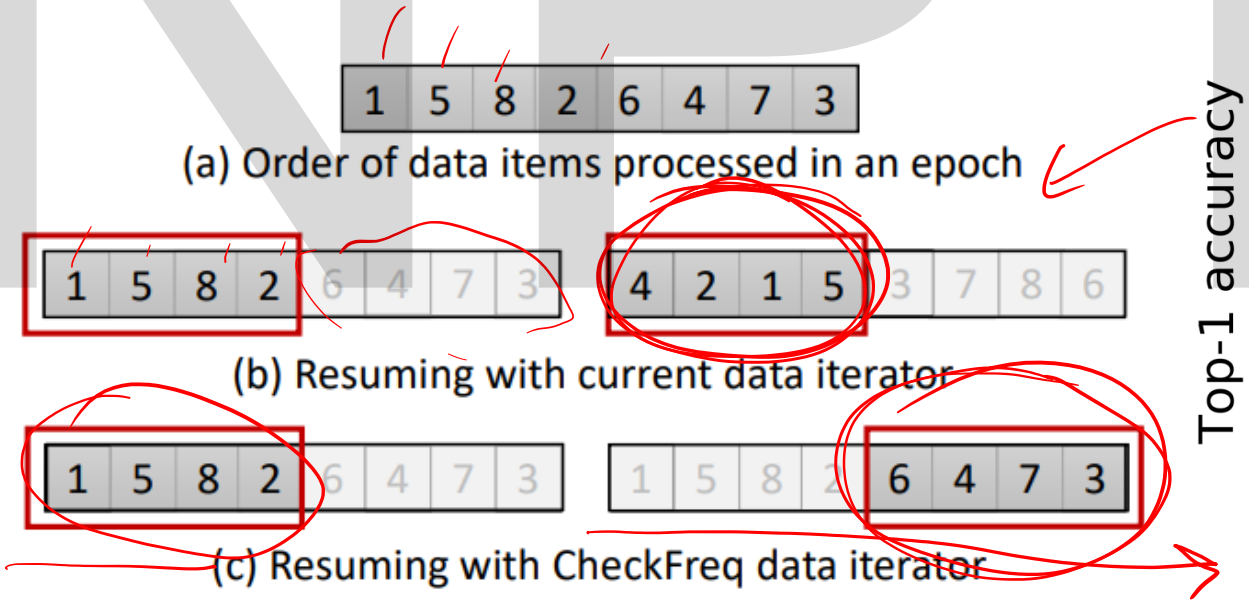
GPU-optimized Snapshots

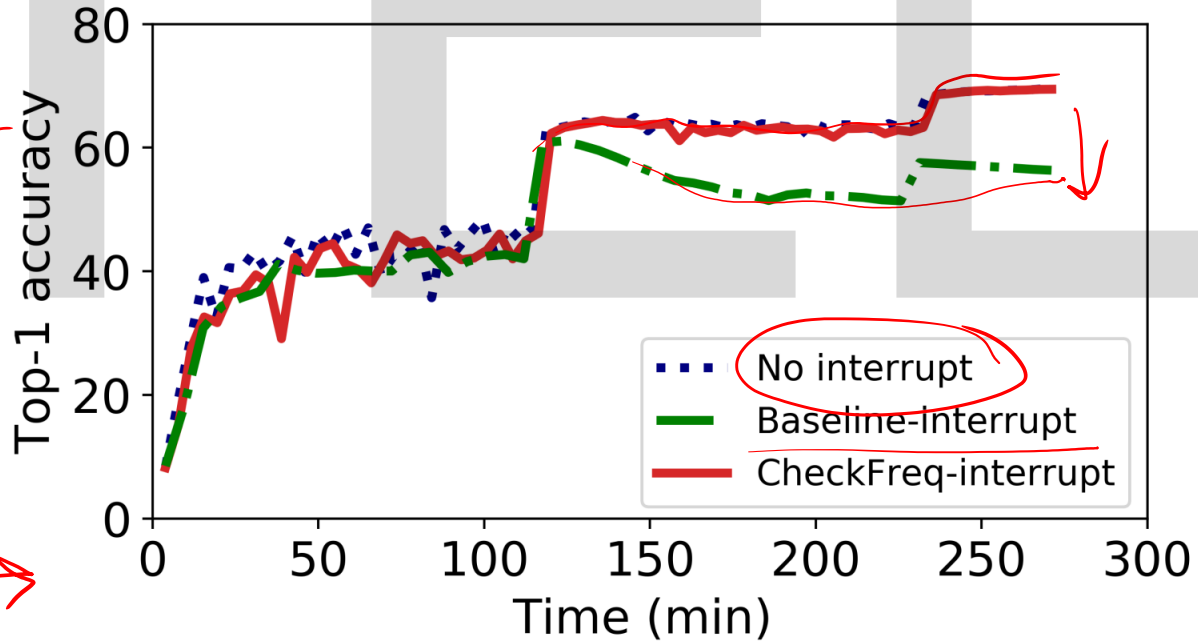
- Cost of serialization and snapshot() is up to 10x lower when done on the GPU
- To further reduce the checkpoint cost, CheckFreq snapshots on the GPU, and asynchronously writes it to CPU memory if it profiles spare memory on the GPU
- If GPU memory is fully utilized, it falls back to pipelined, CPU-side snapshots

NPTEL

Resumable Data Iterator

- **Feeds data item to the GPU process**
- **Has maintain the invariant that each epoch sees a data item exactly once, in a shuffled order**





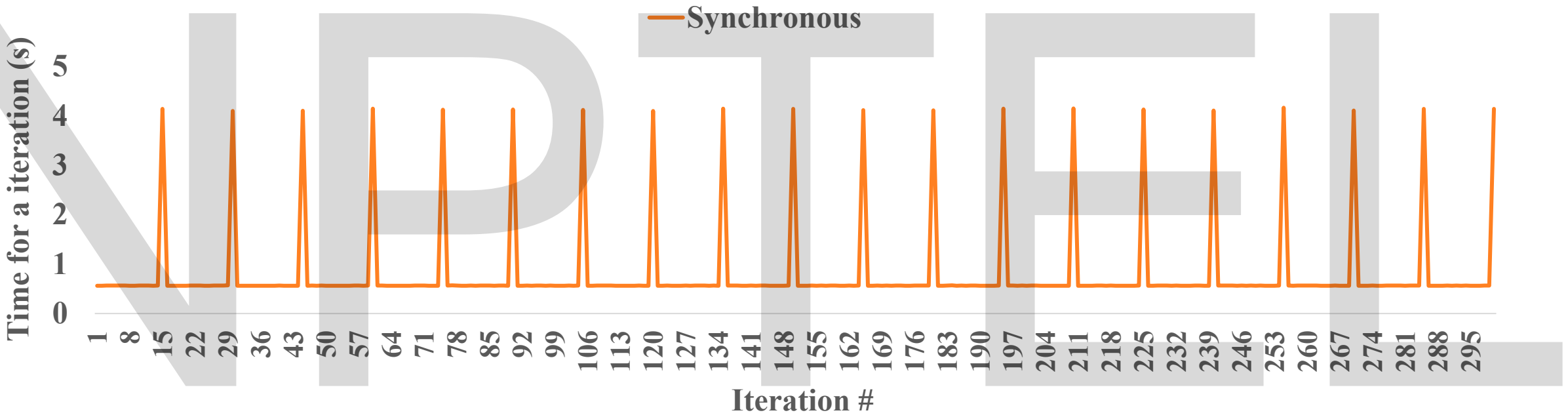
Adaptive Rate Tuning

- Job environment can change, interferences can be introduced
- Run a background process for frequency estimation and modify checkpointing frequency as needed

Setting	Isolated	Static	Adaptive
Overhead	5%	**35%**	5%
Frequency (# iterations)	14	14	19

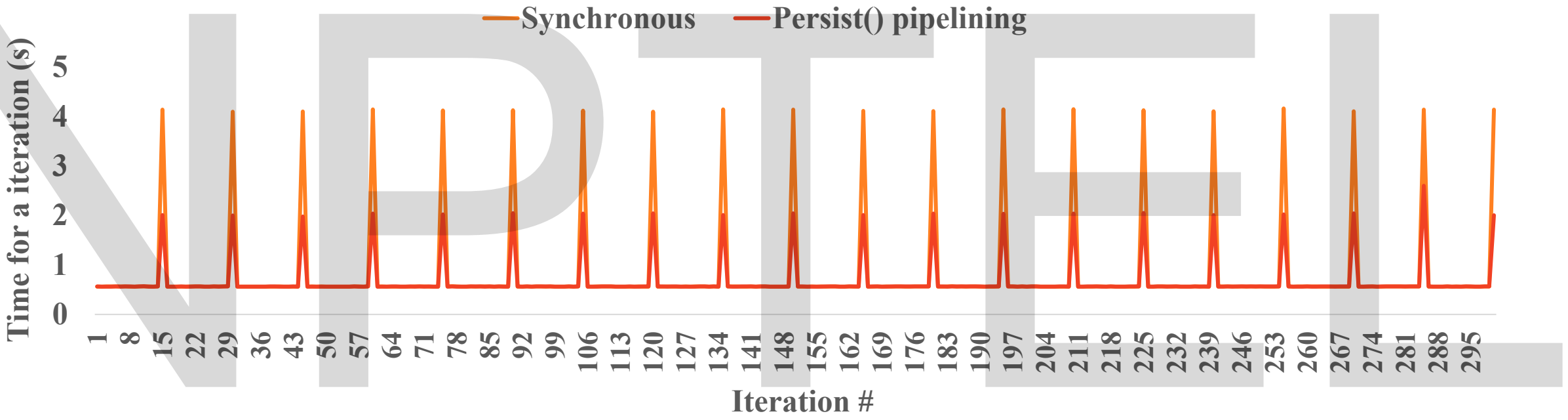
NPTEL

CheckFreq Reduces Checkpoint Stalls



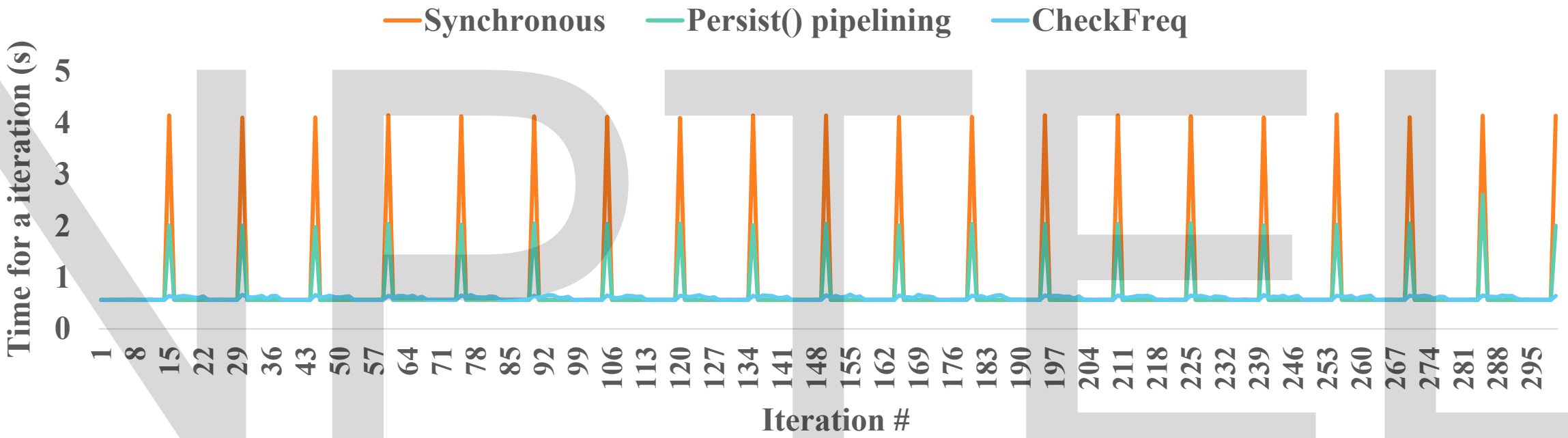
INTEL

CheckFreq Reduces Checkpoint Stalls



Performing asynchronous IO reduces checkpoint cost by 2x but still results in significant stalls

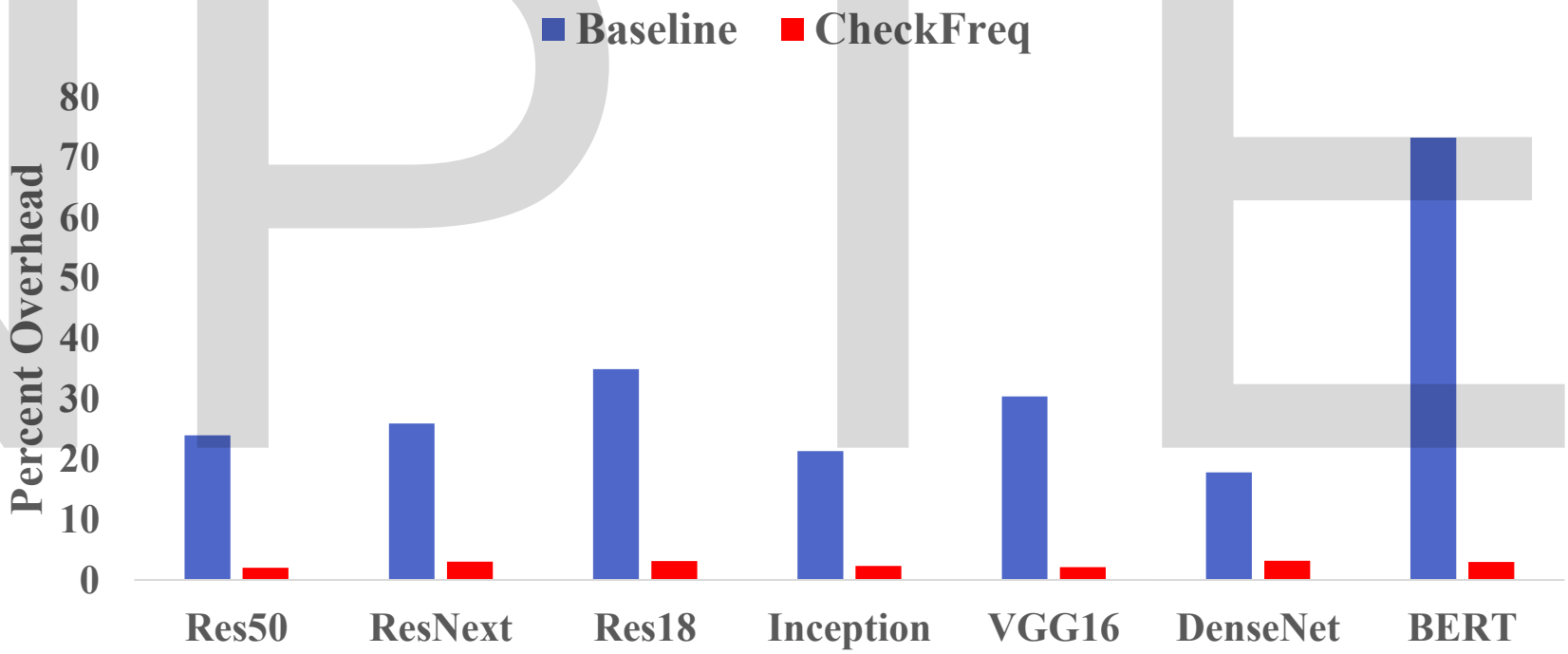
CheckFreq Reduces Checkpoint Stalls



CheckFreq further reduces stalls by carefully pipelining checkpointing with compute

Overall Training Overhead

- When the baseline checkpointing mechanism is performed at a frequency chosen by CheckFreq, it introduces 20 – 70% overhead in training time



NPTEL

CheckFreq Lowers Recovery Time

- **CheckFreq reduces recovery time during an interruption from hours to seconds**

Model	Epoch-based (s)	CheckFreq (s)
Res18	840	5
Res50	2100	24
VGG16	5700	25
ResNext	7080	32
DenseNet	2340	7
Inception	3000	27
BERT	4920	85

NDEL



THANK YOU

Slide Courtesy: Murali Annavaram (University of Southern California), R. Govindarajan (IISc), Arkaprava Basu (IISc), Jayashree Mohan (Microsoft Research)